Algorithm

Binary Tree Node Counting (Modified Version) : five counters, leaf display, recursive and iterative counting

1. Aim of the Algorithm

The aim is to visit every node of a binary tree and update five counters: **total nodes**, **leaf nodes**, **internal nodes**, nodes having **exactly one child** and nodes having **exactly two children**. Whenever a leaf is found, its **value is displayed**. In addition, the total number of nodes is counted again by two separate methods, **recursive** and **iterative (queue)**, and the results are compared.

2. What is New in the Modified Version

Feature	Previous version	Modified version
Two-child nodes	Not counted	Counted with a new counter, twoChildNodes
Leaf nodes	Only the count was shown	The count and the values of all leaf nodes are shown
Total node count	One recursive traversal	Recursive method and iterative method, then compared
Counters	4 counters	5 counters (plus the recursive and iterative totals)
Analysis	Complexity of one method	Time and space of both methods compared

3. Input, Output and Assumptions

Item	Description
Input	Pointer `root` to the root of a binary tree (the tree may be empty).
Output	Leaf values, five node counts, recursive total, iterative total, and a message saying whether the two totals are equal.
Assumption 1	Each node has `data`, `left` and `right`; a missing child is `nullptr` (NULL).
Assumption 2	The tree has no cycles, so every traversal terminates.
Assumption 3	Only the shape of the tree decides the counts; the data values are used only for display.

4. Data Structures and Variables

```
struct Node {
    int data;       // value stored in the node
    Node *left;     // left child  (nullptr if absent)
    Node *right;    // right child (nullptr if absent)
};
```

Variable	Type	Purpose	Initial
`totalNodes`	int (reference)	Counts every node	0
`leafNodes`	int (reference)	Counts nodes with 0 children	0

Variable	Type	Purpose	Initial
`internalNodes`	int (reference)	Counts nodes with 1 or 2 children	0
`oneChildNodes`	int (reference)	Counts nodes with exactly 1 child	0
`twoChildNodes`	int (reference)	Counts nodes with exactly 2 children	0
`queue<Node*> q`	queue	Holds the nodes waiting to be visited (iterative method)	empty
`count`	int	Local counter of the iterative method	0

5. Node Classification Rule

left != NULL	right != NULL	Node type	total	leaf	internal	one	two
No	No	Leaf (value displayed)	+1	+1	0	0	0
Yes	No	One child (left only)	+1	0	+1	+1	0
No	Yes	One child (right only)	+1	0	+1	+1	0
Yes	Yes	Two children	+1	0	+1	0	+1

6. Algorithm 1 : Create the Binary Tree

1. Start the program.
2. Define a node structure containing data, a left child pointer and a right child pointer.
3. Create a new node by allocating memory and initializing its data.
4. Set the left and right child pointers of the new node to NULL.
5. Connect the nodes using their left and right pointers to construct the binary tree.
6. Assign the first node as the root.
7. Pass the root to the counting functions.

7. Algorithm 2 : Count Different Types of Nodes

Step	Action	Explanation
1	Initialize `totalNodes, leafNodes, internalNodes, oneChildNodes, twoChildNodes` to 0	All five counters start at zero before the first call.
2	Call the recursive function with `root`	Traversal begins at the root.
3	`IF node == NULL THEN RETURN`	Base case: an empty subtree has nothing to count.
4	`totalNodes++`	Every non-NULL node is counted once.
5	`IF left == NULL AND right == NULL`	Leaf test. If true: `leafNodes++` and display `node.data`.
6	ELSE `internalNodes++`	At least one child exists.
7	`IF left != NULL AND right != NULL THEN twoChildNodes++`	Both children exist.
8	`ELSE oneChildNodes++`	Internal but not two children, so exactly one child exists.
9	Process the left subtree recursively	Whole left subtree is counted.
10	Process the right subtree recursively	Whole right subtree is counted.
11	Repeat until all nodes are visited	Recursion unwinds when every pointer has been checked.
12	Display all five counters	Done in `main` after the traversal ends.

Design note: the document lists the checks as separate steps (internal, one child, two children). In code they are written as a single `if / else` chain so that each node is classified exactly once and can never be counted in two groups by mistake.

8. Pseudocode for Algorithm 2

```
Algorithm CountTypes(node)
    if node is NULL then return
    totalNodes <- totalNodes + 1
    if node.left is NULL and node.right is NULL then
        leafNodes <- leafNodes + 1
        display node.data
    else
        internalNodes <- internalNodes + 1
        if node.left is not NULL and node.right is not NULL then
            twoChildNodes <- twoChildNodes + 1
        else
            oneChildNodes <- oneChildNodes + 1
```

```
    CountTypes(node.left)
    CountTypes(node.right)
```

9. Dry Run of Algorithm 2

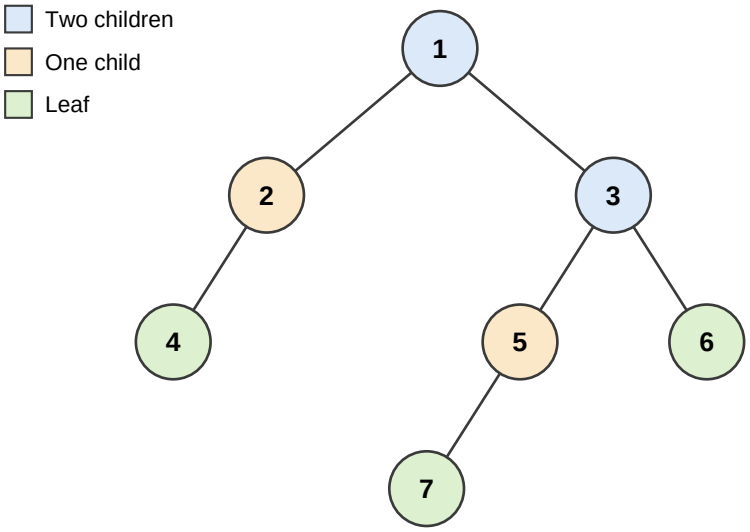

Figure 1: Example binary tree (colour shows the type of each node)

Example tree used in all documents: node 1 is the root with children 2 and 3; node 2 has only a left child (4); node 3 has children 5 and 6; node 5 has only a left child (7).

Nodes are visited in preorder: **1, 2, 4, 3, 5, 7, 6**. Leaf values are displayed at the moment each leaf is found, so they appear as **4 7 6**.

Step	Node	Type	total	leaf	internal	one	two	Leaves shown
1	1	Two children	1	0	1	0	1	(none)
2	2	One child	2	0	2	1	1	(none)
3	4	Leaf	3	1	2	1	1	4
4	3	Two children	4	1	3	1	2	4
5	5	One child	5	1	4	2	2	4
6	7	Leaf	6	2	4	2	2	4 7
7	6	Leaf	7	3	4	2	2	4 7 6

Final result: total = 7, leaf = 3 (values 4, 7, 6), internal = 4, one child = 2 (nodes 2 and 5), two children = 2 (nodes 1 and 3). Checks: total = leaf + internal (7 = 3 + 4); internal = one + two (4 = 2 + 2); leaf = two + 1 (3 = 2 + 1).

10. Algorithm 3 : Recursive Counting Method

1. Start the recursive counting function by passing the root node.
2. If the root is NULL, return zero.
3. Count the current node as one.
4. Recursively count the nodes in the left subtree.
5. Recursively count the nodes in the right subtree.
6. Add the current node count, left subtree count and right subtree count.
7. Return the total number of nodes.

Formula

$N = 1 + N_L + N_R$

Here N is the total number of nodes in the current tree, N_L is the number of nodes in the left subtree and N_R is the number of nodes in the right subtree. If the current tree is empty, its node count is zero.

```
int countRecursive(Node* root) {
    if (root == nullptr)
        return 0;
    return 1 + countRecursive(root->left) + countRecursive(root->right);
}
```

Values returned by the calls on the example tree (calls finish from the bottom upwards):

Node	N_L (left)	N_R (right)	$N = 1 + N_L + N_R$
4	0	0	1
2	1 (node 4)	0	2
7	0	0	1
5	1 (node 7)	0	2
6	0	0	1
3	2 (subtree of 5)	1 (node 6)	4
1	2 (subtree of 2)	4 (subtree of 3)	7

11. Algorithm 4 : Iterative Counting Method Using a Queue

1. Start the iterative counting function.
2. If the root is NULL, return zero.
3. Create an empty queue.
4. Insert the root node into the queue.
5. Initialize the total node counter to zero.
6. Repeat while the queue is not empty.
7. Remove the front node from the queue.
8. Increment the total node counter by one.
9. If the removed node has a left child, insert the left child into the queue.
10. If the removed node has a right child, insert the right child into the queue.
11. Continue until the queue becomes empty.
12. Return the total number of nodes.

This method uses level-order traversal, in which nodes are visited level by level.

```
int countIterative(Node* root) {
    if (root == nullptr) return 0;
    queue<Node*> q;
    q.push(root);
    int count = 0;
    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        count++;
        if (current->left  != nullptr) q.push(current->left);
        if (current->right != nullptr) q.push(current->right);
    }
    return count;
}
```

Node popped	Queue after pushing children	count
1	[2, 3]	1
2	[3, 4]	2
3	[4, 5, 6]	3
4	[5, 6]	4
5	[6, 7]	5
6	[7]	6
7	[ ] (empty)	7

12. Algorithm 4B : Iterative Counting Using a Stack (Alternative)

The same idea works with a stack. Push the root; then repeat: pop a node, count it, push its right child and then its left child (so that the left child is processed first). This visits nodes in preorder, so it is the iterative form of the recursion.

Node popped	Stack after pushing children (top at right)	count
1	[3, 2]	1
2	[3, 4]	2
4	[3]	3
3	[6, 5]	4
5	[6, 7]	5
7	[6]	6
6	[ ] (empty)	7

The pop order 1, 2, 4, 3, 5, 7, 6 is the same preorder sequence as the recursive method, and the final count is again 7.

13. Complexity Analysis

Method	Time	Auxiliary space	Reason
Algorithm 2 (types + leaf display)	O(n)	O(h)	Each node is visited once; recursion depth equals the height h.
Recursive count	O(n)	O(h)	One call per node plus one per NULL pointer (2n + 1 calls); stack depth is h.
Iterative count (queue)	O(n)	O(w)	Each node is pushed and popped once; the queue holds at most one level, w = maximum width.
Iterative count (stack)	O(n)	O(h)	Each node is pushed and popped once; the stack holds about one root-to-leaf path of pending nodes.

Here n is the number of nodes, h the height of the tree and w its maximum width. For a balanced tree h is about log n, for a skewed tree h = n, and for a perfect tree w is about n / 2.

14. Recursive versus Iterative

Feature	Recursive method	Iterative method (queue)	Iterative method (stack)
Technique	Function calls itself for left and right subtrees	Loop with a queue of nodes still to visit	Loop with a stack of nodes still to visit
Traversal order	Root, Left, Right (preorder style)	Level by level (level order)	Root, Left, Right (if right child is pushed first)
Stopping condition	Node is NULL (returns 0)	Queue becomes empty	Stack becomes empty
Time complexity	O(n)	O(n)	O(n)
Auxiliary space	O(h) call stack	O(w) queue, w = maximum width of the tree	O(h) explicit stack
Skewed tree (h = n)	O(n) stack depth	O(1): queue holds at most one node	O(1): stack holds at most one node
Perfect tree	O(log n)	About n/2 nodes, so O(n)	O(log n)
Stack overflow risk	Yes, for very deep trees	No (memory comes from the heap)	No (memory comes from the heap)
Code length	Shortest and simplest	Longer; needs queue header	Longer; needs stack header

Feature	Recursive method	Iterative method (queue)	Iterative method (stack)
Function calls / loop passes (n = 7)	15 calls (2n + 1)	7 loop passes	7 loop passes
Result for example tree	7	7	7

15. Edge Cases

Tree	total	leaf	internal	one	two	Leaves displayed
Empty tree	0	0	0	0	0	(none)
Single node (value 10)	1	1	0	0	0	10
Left-skewed, 4 nodes (4 3 2 1)	4	1	3	3	0	1
Right-skewed, 4 nodes (1 2 3 4)	4	1	3	3	0	4
Perfect tree, 7 nodes (4 2 6 1 3 5 7)	7	4	3	0	3	1 3 5 7

16. Common Mistakes to Avoid

- Forgetting the NULL base case, which gives infinite recursion.
- Passing counters by value instead of by reference, so the results stay 0.
- Using separate `if` statements for one-child and two-child tests without an `else`, so a node can be counted twice.
- Printing a leaf value but forgetting `leafNodes++` (or the other way round).
- Printing the heading `Leaf Nodes:` after the function call instead of before it, so the values appear in the wrong place.
- Pushing a NULL child into the queue, which crashes when the node is popped and its pointers are used.
- Assuming the leaf display order is the same for every traversal: preorder gives 4 7 6, level order would give 4 6 7.

17. Summary

The modified algorithm classifies each node as a leaf, one-child or two-child node in one preorder traversal, displays the leaf values as they are found, and then verifies the total node count with two independent methods. Both counting methods take O(n) time; they differ in auxiliary space (call stack O(h) against queue O(w)), so the choice depends on the shape of the tree.